2. Builder Design Pattern in Java

2.1 Introduction

The Builder Design Pattern is a creational design pattern that allows for the step-by-step construction of complex objects. It helps separate the construction logic from the object itself, providing flexibility and readability.

2.2 The Problem with Telescoping Constructors

In Java, constructing objects with many optional parameters can lead to an explosion of constructors:

```
public class User {
    private String name;
    private int age;
    private String email;

    public User(String name) {
        this(name, 0, null);
    }

    public User(String name, int age) {
        this(name, age, null);
    }

    public User(String name, int age, String email) {
        this.name = name;
        this.age = age;
        this.email = email;
    }
}
```

Problems:

- Hard to read and maintain
- Difficult to know which constructor to use

2.3 Builder Pattern - Basic Concept

The Builder pattern solves this by:

- Creating a static inner class Builder
- Providing setter methods that return the builder itself (for chaining)
- A final build() method that creates the actual object

2.4 Step-by-Step Examples

Example1: Beginner: Simple User Class

```
package com.mannemlokesh;

public class User {
    private final String name;
    private final int age;
    private final String email;

    private User(Builder builder) {
        this.name = builder.name;
        this.age = builder.age;
        this.email = builder.email;
    }

    public static class Builder {
        private String name;
        private int age;
        private String email;

        public Builder setName(String name) {
            this.name = name;
            return this;
        }

        public Builder setAge(int age) {
            this.age = age;
            return this;
        }

        public Builder setEmail(String email) {
            this.email = email;
            return this;
        }

        public User build() {
            return new User(this);
        }
    }

    public String toString() {
        return "User{name='" + name + "', age=" + age + ", email='" + email +
                                                                    "'}";
    }

    public static void main(String[] args) {
        User user = new User.Builder()
            .setName("Alice")
            .setAge(25)
            .setEmail("alice@example.com")
            .build();

        System.out.println(user);
    }
}
```

Output:

```
User{name='Alice', age=25, email='alice@example.com'}
```

Example2: Intermediate: Validation and Fluent Interface

```
package com.mannemlokesh;

public class Product {
    private final String name;
    private final double price;

    private Product(Builder builder) {
        this.name = builder.name;
        this.price = builder.price;
    }

    public static class Builder {
        private String name;
        private double price;

        public Builder setName(String name) {
            this.name = name;
            return this;
        }

        public Builder setPrice(double price) {
            this.price = price;
            return this;
        }

        public Product build() {
            if (name == null || name.isEmpty()) {
                throw new IllegalArgumentException("Product name cannot be
                                                                    empty");
            }
            if (price <= 0) {
                throw new IllegalArgumentException("Price must be greater than
                                                                     zero");
            }
            return new Product(this);
        }
    }

    public String toString() {
        return "Product{name='" + name + "', price=" + price + "}";
    }

    public static void main(String[] args) {
        Product p = new Product.Builder()
            .setName("Laptop")
            .setPrice(999.99)
            .build();

        System.out.println(p);
    }
}
```

Output:

```
Product{name='Laptop', price=999.99}
```

Example3: Advanced: Nested Builders & Real-World Example

```
package com.mannemlokesh;

public class Car {
    private final String model;
    private final Engine engine;

    private Car(Builder builder) {
        this.model = builder.model;
        this.engine = builder.engine;
    }

    public static class Builder {
        private String model;
        private Engine engine;

        public Builder setModel(String model) {
            this.model = model;
            return this;
        }

        public Builder setEngine(Engine engine) {
            this.engine = engine;
            return this;
        }

        public Car build() {
            return new Car(this);
        }
    }

    public static class Engine {
        private final int horsepower;

        public Engine(int horsepower) {
            this.horsepower = horsepower;
        }

        public String toString() {
            return horsepower + " HP";
        }
    }

    public String toString() {
        return "Car{model='" + model + "', engine=" + engine + "}";
    }

    public static void main(String[] args) {
        Car.Engine engine = new Car.Engine(300);
        Car car = new Car.Builder()
            .setModel("Tesla Model S")
            .setEngine(engine)
            .build();
```

```
        System.out.println(car);
    }
}
```

Output:

```
Car{model='Tesla Model S', engine=300 HP}
```

2.5 Builder Pattern in Java 17+ (Using Records)

Example5:

```
package com.mannemlokesh;

public record Book(String title, String author, int year) {
    public static class Builder {
        private String title;
        private String author;
        private int year = 2024;

        public Builder setTitle(String title) {
            this.title = title;
            return this;
        }

        public Builder setAuthor(String author) {
            this.author = author;
            return this;
        }

        public Builder setYear(int year) {
            this.year = year;
            return this;
        }

        public Book build() {
            return new Book(title, author, year);
        }
    }

    public static void main(String[] args) {
        Book book = new Book.Builder()
            .setTitle("Effective Java")
            .setAuthor("Joshua Bloch")
            .setYear(2018)
            .build();

        System.out.println(book);
    }
}
```

Output:

```
Book[title=Effective Java, author=Joshua Bloch, year=2018]
```

2.6 When to Use / Not to Use

✅ **Use When:**

- You have objects with many optional parameters
- Immutability and validation are required
- You want clean, readable code with fluent chaining

❌ **Avoid When:**

- You have simple POJOs with only 1–2 fields
- Performance is extremely critical (builder adds slight overhead)

2.7 Summary

Feature	Benefit
Fluent setters	Improves readability
Immutability	Thread safety and integrity
Centralized validation	Cleaner and safer construction
Separation of concern	Better code organization

The Builder Pattern is especially useful in real-world domains like:

- Configuration objects
- Complex UI components
- Request/response builders in APIs
- Domain modeling (Car, House, Product, etc.)

By following this pattern, you can write Java code that is robust, scalable, and easier to maintain.